

LAPORAN DATA CLEANING SHIPPING LOGISTICS

Pembersihan dan Persiapan Data Pengiriman

Periode Januari 2023 sampai Desember 2025

Eko Damar Yogi

PORTOFOLIO ANALISIS DATA

Microsoft SQL Server dan T-SQL

DAFTAR ISI

DAFTAR ISI	2
RINGKASAN.....	3
1 PENDAHULUAN	4
2 DESKRIPSI DATASET	5
2 DESKRIPSI DATASET LANJUTAN	6
3 METODOLOGI DATA CLEANING	7
4 TAHAPAN DATA CLEANING DAN PREPARATION.....	8
4 TAHAPAN DATA CLEANING LANJUTAN	9
4 TAHAPAN DATA CLEANING LANJUTAN	10
5 HASIL DAN VALIDASI KUALITAS DATA	11
6 INTERPRETASI MASALAH DATA	12
7 BATASAN DOKUMENTASI	13
8 KESIMPULAN	14
9 LAMPIRAN TEKNIS	15

RINGKASAN

Proses data cleaning Shipping Logistics mencakup pemeriksaan nilai kosong, penanganan data kategorikal dan numerik, deduplikasi berdasarkan OrderID, serta standarisasi nama kota. File TableCleaning.sql menjadi acuan utama untuk menjelaskan tindakan yang dilakukan pada database LogistikDB. Tabel sumber disalin ke tabel kerja sebelum hasilnya disimpan sebagai dbo.ShippingLogistics_Cleaned.

Penanganan nilai kosong mengikuti karakteristik kolom. DelayReason yang NULL pada status Delivered diberi label No Delay. VehicleType dan PaymentMethod diberi label Unknown, sedangkan FuelSurchargeIDR diisi median global dari nilai yang tersedia. DeliveryDate, ActualDeliveryDays, dan CustomerRating dipertahankan NULL sesuai keputusan yang didokumentasikan dalam skrip dan laporan cleaning sebelumnya.

Komentar SQL dan laporan cleaning sebelumnya mencatat pola kekosongan bersama pada 120 baris untuk VehicleType, PaymentMethod, dan FuelSurchargeIDR. Keduanya juga menyebut dua order duplikat dengan biaya berbeda. Aturan deduplikasi mempertahankan satu baris per OrderID dengan ShippingCostIDR terendah. Angka tersebut merupakan temuan terdokumentasi, bukan hasil eksekusi ulang pada penyusunan laporan ini.

Laporan analisis mencatat hasil cleaning sebanyak 6.000 baris dan 24 kolom. Sebanyak 193 nilai DeliveryDate, 193 nilai ActualDeliveryDays, dan 714 nilai CustomerRating masih kosong. Nilai kosong yang dipertahankan tidak boleh diubah maknanya menjadi nol, tanggal buatan, atau rating pelanggan yang tidak pernah diberikan.

Dua keputusan cleaning memerlukan perhatian saat data digunakan. Label No Delay tidak membuktikan ketepatan waktu karena status Delivered hanya menunjukkan hasil pengiriman. Selain itu, biaya terendah pada kelompok duplikat merupakan aturan pemilihan record yang belum dikonfirmasi terhadap bukti transaksi sumber.

Dokumentasi ini memperbaiki penyebutan jumlah kolom dan penafsiran hasil pada laporan sebelumnya, sambil mempertahankan kesesuaian dengan SQL yang dilampirkan. Pemeriksaan tambahan disajikan sebagai rekomendasi, sehingga tidak tercampur dengan langkah yang sudah tercantum dalam skrip.

Sumber: TableCleaning.sql

1 PENDAHULUAN

1.1 Latar belakang

Dataset pengiriman digunakan untuk menilai volume, durasi, biaya, dan kualitas layanan logistik. Nilai kosong, pengulangan order, serta penulisan kota yang tidak konsisten dapat mengubah hasil penghitungan dan pengelompokan. Karena itu, data perlu dipersiapkan sebelum masuk ke tahap analisis.

Laporan sebelumnya menjelaskan bahwa sumber data berupa CSV yang diimpor ke Microsoft SQL Server. Skrip yang tersedia dimulai ketika tabel `dbo.ShippingLogistics` sudah berada dalam `LogistikDB`. Dengan demikian, proses impor merupakan konteks proyek; langkah impor dan pengaturan tipe kolom tidak tercantum dalam file SQL ini.

1.2 Tujuan pekerjaan

Pekerjaan ini bertujuan mendokumentasikan pemeriksaan kualitas data dan aturan transformasi yang diterapkan, menjelaskan alasan mempertahankan atau mengisi nilai kosong, serta memperjelas batas penggunaan tabel hasil cleaning untuk analisis berikutnya. Setiap keputusan dijelaskan melalui operasi SQL yang dapat ditelusuri.

1.3 Pertanyaan pemeriksaan data

- Kolom mana yang memiliki NULL dan bagaimana hubungannya dengan status pengiriman?
- Bagaimana menangani informasi yang tidak tercatat tanpa menciptakan tanggal, durasi, atau rating buatan?
- Bagaimana memilih record ketika satu OrderID muncul lebih dari sekali?
- Bagaimana menstandarisasi kota agar pengelompokan rute dan tujuan konsisten?
- Bukti apa yang tersedia untuk menyatakan kondisi data setelah cleaning?

1.4 Ruang lingkup dan dasar penyusunan

Ruang lingkup utama adalah urutan tindakan dalam `TableCleaning.sql`. Laporan cleaning digunakan sebagai konteks keputusan, sedangkan laporan analisis digunakan sebagai acuan format serta sumber sekunder untuk profil tabel akhir. Database, CSV, dan notebook tidak dilampirkan pada permintaan ini; laporan tidak menyatakan bahwa SQL telah dieksekusi ulang.

Sumber: TableCleaning.sql

2 DESKRIPSI DATASET

2.1 Sumber data dan alur tabel

Data awal disimpan pada `dbo.ShippingLogistics`. Seluruh UPDATE dan DELETE diarahkan ke `dbo.ShippingLogistics_Cleaning`. Setelah standardisasi, tabel kerja disalin ke `dbo.ShippingLogistics_Cleaned` sebagai keluaran untuk analisis.

Tabel 1 Profil data dan sumber keterangannya

Aspek	Keterangan
Database	LogistikDB
Tabel sumber	<code>dbo.ShippingLogistics</code>
Tabel kerja	<code>dbo.ShippingLogistics_Cleaning</code>
Tabel akhir	<code>dbo.ShippingLogistics_Cleaned</code>
Jumlah baris akhir	6.000 menurut laporan analisis halaman 5
Jumlah atribut	24 menurut daftar atribut dan laporan analisis halaman 6
Periode	Januari 2023–Desember 2025 menurut laporan analisis
Jumlah baris awal dan terhapus	Tidak tersedia pada keluaran eksekusi yang dilampirkan

2.2 Granularitas data

ShipmentSK merupakan identitas baris, sedangkan OrderID menjadi kunci pengelompokan deduplikasi. Skrip menyisakan satu record untuk setiap kelompok OrderID. Aturan ini mengasumsikan satu order cukup diwakili satu baris; jika proses bisnis mengizinkan beberapa shipment untuk satu order, granularitas harus dikonfirmasi sebelum penghapusan diterapkan pada data lain.

2.3 Koreksi jumlah atribut

Laporan cleaning sebelumnya menyebut 23 kolom, tetapi daftar yang ditampilkan berisi 24 atribut. Laporan analisis juga mencatat 24 kolom pada saat pemuatan data. Laporan ini menggunakan 24 sebagai jumlah atribut yang terdokumentasi. Tipe fisik SQL Server tidak ditetapkan ulang karena DDL tabel tidak disertakan.

Sumber: TableCleaning.sql

2 DESKRIPSI DATASET LANJUTAN

2.4 Struktur atribut

Tabel 2 Struktur 24 atribut

Atribut	Makna informasi
ShipmentSK	Identitas baris pengiriman
OrderID	Identitas order dan kunci deduplikasi
OrderDate	Tanggal order
PickupDate	Tanggal pengambilan barang
DeliveryDate	Tanggal realisasi pengiriman
OriginCity	Kota asal
DestinationCity	Kota tujuan
DistanceKM	Jarak dalam kilometer
WeightKG	Berat dalam kilogram
VolumeM3	Volume dalam meter kubik
Carrier	Mitra pengiriman
ShippingType	Jenis layanan
VehicleType	Jenis kendaraan
CustomerType	Jenis pelanggan
PaymentMethod	Metode pembayaran
WarehouseID	Identitas gudang
DriverID	Identitas pengemudi
ShippingCostIDR	Komponen biaya pengiriman dalam rupiah
FuelSurchargeIDR	Tambahan biaya bahan bakar dalam rupiah
PlannedDeliveryDays	Durasi rencana dalam hari
ActualDeliveryDays	Durasi aktual dalam hari
DeliveryStatus	Delivered, Delayed, Returned, atau Lost
DelayReason	Alasan gangguan atau label hasil imputasi
CustomerRating	Rating pelanggan yang tercatat

3 METODOLOGI DATA CLEANING

3.1 Tools dan prinsip kerja

Skrip menggunakan T-SQL di Microsoft SQL Server. Query SELECT dipakai untuk inspeksi, dynamic SQL untuk menghitung NULL per kolom, UPDATE untuk imputasi dan standarisasi, serta ROW_NUMBER dengan DELETE untuk deduplikasi. Tabel sumber tetap tersedia karena perubahan data dilakukan pada salinannya.

Tabel 3 Urutan operasi pada skrip

Tahap	Operasi utama	Tujuan
1	USE dan SELECT	Memilih database dan meninjau data
2	DROP dan SELECT INTO	Membuat ulang tabel kerja
3	STRING_AGG dan IS NULL	Memeriksa NULL pada seluruh kolom
4	GROUP BY dan UPDATE	Menangani DelayReason
5	SELECT dan agregasi kondisi	Menelusuri NULL terkait status dan tiga kolom
6	UPDATE dan PERCENTILE_CONT	Mengisi kategori dan biaya bahan bakar
7	ROW_NUMBER dan DELETE	Menyisakan satu baris per OrderID
8	TRIM dan UPPER	Menstandarisasi kedua kolom kota
9	DROP dan SELECT INTO	Membentuk tabel akhir

3.2 Urutan transformasi dan implikasinya

Imputasi FuelSurchargeIDR dilakukan sebelum deduplikasi. Dengan urutan ini, nilai median dihitung dari tabel kerja yang masih dapat memuat record duplikat. Jika duplikasi mengubah distribusi biaya, hasil median dapat berbeda dari median pada data yang sudah unik. Laporan mempertahankan urutan tersebut sebagai deskripsi skrip asli.

3.3 Dasar pembuktian

Operasi SQL menjadi bukti tentang aturan yang diterapkan. Komentar SQL dan laporan lama menjadi bukti temuan yang pernah didokumentasikan. Angka profil tabel akhir bersumber dari laporan analisis. Tanpa hasil query langsung, ketiga jenis bukti tersebut tidak disamakan dengan verifikasi ulang terhadap database.

Sumber: TableCleaning.sql.

4 TAHAPAN DATA CLEANING DAN PREPARATION

4.1 Pemeriksaan awal dan tabel kerja

Skrip menampilkan sampel TOP 10, menghitung total baris, dan menghitung OrderID berbeda. Alias pada dua query awal tertukar maknanya: COUNT(*) diberi nama total_OrderID_Unique, sedangkan COUNT(DISTINCT OrderID) diberi nama total_rows. Penafsiran harus mengikuti fungsi agregasinya; COUNT(DISTINCT OrderID) juga tidak menghitung NULL.

```
DROP TABLE IF EXISTS dbo.ShippingLogistics_Cleaning;  
SELECT * INTO dbo.ShippingLogistics_Cleaning  
FROM dbo.ShippingLogistics;
```

Perintah DROP membuat ulang tabel kerja jika tabel tersebut sudah ada. Tabel sumber tidak dihapus oleh skrip, tetapi perubahan terdahulu pada tabel kerja tidak dipertahankan ketika seluruh rangkaian dijalankan kembali.

4.2 Pemeriksaan NULL pada seluruh kolom

INFORMATION_SCHEMA.COLUMNS memilih kolom pada schema dbo dan tabel ShippingLogistics_Cleaning. STRING_AGG menyusun query COUNT(*) WHERE kolom IS NULL, menggabungkannya dengan UNION ALL, lalu sp_executesql menjalankan rangkaian tersebut. QUOTENAME digunakan untuk membatasi nama kolom sebagai identifier SQL.

Pemeriksaan ini menghitung NULL, bukan string kosong, teks berisi spasi, atau penanda seperti N/A. Sampel TOP 10 membantu inspeksi awal, tetapi tidak menggantikan penghitungan terhadap seluruh tabel.

4.3 Imputasi DelayReason

```
UPDATE dbo.ShippingLogistics_Cleaning  
SET DelayReason = 'No Delay'  
WHERE DelayReason IS NULL AND DeliveryStatus = 'Delivered';
```

Sebelum UPDATE, skrip menghitung NULL DelayReason menurut DeliveryStatus. Komentar menyatakan seluruh kekosongan yang ditemukan terkait Delivered. Setelah UPDATE, tersedia query untuk menghitung sisa NULL DelayReason, tetapi hasil angkanya tidak terlampir.

No Delay merupakan label imputasi alasan, bukan hasil pengujian SLA. Ketepatan waktu tetap harus diperiksa melalui ActualDeliveryDays dan PlannedDeliveryDays. Penafsiran ini juga konsisten dengan laporan analisis yang menemukan pengiriman Delivered dengan durasi aktual melebihi rencana.

Sumber: TableCleaning.sql

4 TAHAPAN DATA CLEANING LANJUTAN

4.4 DeliveryDate dan ActualDeliveryDays

Query menampilkan DeliveryDate, ActualDeliveryDays, dan DeliveryStatus untuk record dengan DeliveryDate IS NULL. Komentar SQL dan laporan lama mengaitkan kekosongan tersebut dengan Lost. Kedua kolom tidak diimputasi karena tidak tersedia realisasi pengantaran yang dapat diisikan.

Filter ini belum menjadi pemeriksaan independen atas seluruh NULL ActualDeliveryDays. Validasi tambahan diperlukan untuk mendeteksi kasus durasi kosong saat tanggal terisi, atau ketidaksesuaian sebaliknya. Untuk analisis durasi, NULL harus tetap dibedakan dari nol hari.

4.5 VehicleType dan PaymentMethod

Agregasi SUM(CASE WHEN ... THEN 1 ELSE 0 END) menghitung NULL masing-masing kolom VehicleType, PaymentMethod, dan FuelSurchargeIDR serta irisan ketiganya. Komentar SQL mencatat 120 baris yang sama-sama kosong; penyebab operasionalnya belum dibuktikan melalui log sistem.

```
UPDATE dbo.ShippingLogistics_Cleaning
SET VehicleType = 'Unknown', PaymentMethod = 'Unknown'
WHERE VehicleType IS NULL OR PaymentMethod IS NULL;
```

Kedua kolom diberi label Unknown untuk mempertahankan record dengan kategori tidak diketahui. Karena kondisi memakai OR dan SET mengubah dua kolom sekaligus, query juga menimpa nilai terisi jika hanya salah satu kolom kosong. Dampak ini perlu diperhatikan ketika pola kekosongan tidak lagi berimpit; pembaruan per kolom lebih aman untuk penggunaan ulang.

4.6 FuelSurchargeIDR dan CustomerRating

FuelSurchargeIDR diperiksa menurut status; komentar menyebut Delayed, Delivered, dan Returned. NULL kemudian diisi menggunakan PERCENTILE_CONT(0.5) atas seluruh nilai FuelSurchargeIDR yang tidak kosong. Median yang digunakan bersifat global, tanpa pengelompokan kendaraan, carrier, atau layanan.

Median membatasi pengaruh nilai ekstrem dibanding rata-rata, tetapi tetap merupakan estimasi biaya yang tidak tercatat. Nilai median numeriknya tidak tersedia dalam berkas SQL. CustomerRating tetap NULL sesuai komentar yang menyatakan bahwa feedback pelanggan bersifat opsional; tidak ada UPDATE untuk kolom ini.

Sumber: TableCleaning.sql

4 TAHAPAN DATA CLEANING LANJUTAN

4.7 Deteksi dan penghapusan duplikasi

OrderID dengan COUNT(*) > 1 digunakan untuk menampilkan kelompok record berulang. Komentar SQL menyatakan sebagian besar record memiliki nilai sama dan dua order mempunyai ShippingCostIDR berbeda. Tidak tersedia keluaran rinci per order untuk memeriksa ulang perbedaan tersebut.

```
WITH DuplicateCheck AS (  
    SELECT *, ROW_NUMBER() OVER (  
        PARTITION BY OrderID  
        ORDER BY ShippingCostIDR ASC  
    ) AS row_num  
    FROM dbo.ShippingLogistics_Cleaning  
)  
DELETE FROM DuplicateCheck WHERE row_num > 1;
```

Aturan berlaku pada semua kelompok OrderID, bukan hanya baris yang identik di seluruh kolom. Record berbiaya terendah dipertahankan. Biaya yang lebih rendah belum membuktikan nilai transaksi yang benar. Jika biaya sama tetapi atribut lain berbeda, skrip tidak menetapkan pemilih tambahan sehingga baris yang bertahan tidak ditentukan secara unik oleh aturan pengurutan.

Sesudah DELETE, GROUP BY OrderID HAVING COUNT(*) > 1 digunakan untuk mencari duplikasi tersisa. Hasil ideal adalah tidak ada baris yang dikembalikan; keluaran aktual query tidak disertakan.

4.8 Standardisasi nama kota

```
UPDATE dbo.ShippingLogistics_Cleaning  
SET OriginCity = TRIM(UPPER(OriginCity)),  
    DestinationCity = TRIM(UPPER(DestinationCity));
```

UPPER menyamakan kapitalisasi, sedangkan TRIM menghapus spasi di tepi teks. Transformasi membantu pengelompokan kota. Salah eja, singkatan berbeda, dan spasi berulang di tengah nama tidak otomatis diselesaikan oleh dua fungsi tersebut.

4.9 Pembentukan tabel akhir

Skrip menghapus tabel dbo.ShippingLogistics_Cleaned jika sudah ada, kemudian membuatnya kembali melalui SELECT * INTO dari tabel kerja. Tidak ada penambahan kolom turunan, perubahan tipe melalui ALTER COLUMN, ataupun penerapan constraint unik secara eksplisit pada skrip ini.

Sumber: TableCleaning.sql

5 HASIL DAN VALIDASI KUALITAS DATA

5.1 Profil hasil yang terdokumentasi

Laporan analisis mencatat 6.000 baris dan 24 kolom saat tabel hasil cleaning dimuat. Angka berikut diambil dari laporan tersebut sebagai dokumentasi kondisi akhir, bukan dihitung ulang dari database pada penyusunan laporan ini.

Tabel 4 Kelengkapan kolom setelah cleaning

Kolom	Terisi	NULL	Keterangan
DeliveryDate	5.807	193	Dipertahankan tanpa tanggal buatan
ActualDeliveryDays	5.807	193	Durasi tidak tersedia tetap kosong
CustomerRating	5.286	714	Hanya rating tercatat yang dianalisis
21 kolom lainnya	6.000 per kolom	0	Menurut profil di laporan analisis

5.2 Cakupan validasi pada skrip

Tabel 5 Pemeriksaan dan batas pembuktiannya

Pemeriksaan	Tersedia dalam SQL	Batas informasi
Jumlah data awal	COUNT(*) dan COUNT DISTINCT	Angka keluaran tidak dilampirkan
NULL semua kolom	Dynamic SQL sebelum imputasi	Tidak diulang di akhir skrip
NULL DelayReason	COUNT setelah UPDATE	Angka keluaran tidak dilampirkan
Duplikasi OrderID	GROUP BY setelah DELETE	Hasil kosong belum terlampir
Pemeriksaan akhir	TOP 10 sebelum tabel final	Sampel tidak memvalidasi seluruh data

5.3 Rekonsiliasi yang masih diperlukan

Audit berikutnya perlu mencatat jumlah baris sumber, jumlah baris terhapus, dan jumlah baris akhir; menjalankan ulang pemeriksaan NULL seluruh kolom; serta memeriksa jumlah OrderID berbeda dan OrderID NULL. Angka 120 baris sebelum deduplikasi tidak otomatis sama dengan jumlah record hasil imputasi yang masih bertahan pada tabel akhir.

Sumber: TableCleaning.sql

6 INTERPRETASI MASALAH DATA

6.1 Ringkasan aturan dan implikasinya

Tabel 6 Masalah data dan perlakuan yang diterapkan

Kolom	Perlakuan	Implikasi untuk analisis
DelayReason	No Delay pada NULL dan Delivered	Label alasan tidak mengukur ketepatan waktu
DeliveryDate dan ActualDeliveryDays	NULL dipertahankan	Durasi hanya dianalisis ketika tersedia
VehicleType dan PaymentMethod	Unknown	Kategori tidak diketahui tetap terlihat
FuelSurchargeIDR	Median global	Sebagian biaya merupakan nilai imputasi
CustomerRating	NULL dipertahankan	Rata-rata mewakili pelanggan yang memberi rating
OrderID	Satu baris dengan biaya terendah	Bergantung pada asumsi satu order satu record
OriginCity dan DestinationCity	TRIM dan UPPER	Format seragam belum menjamin nama kota benar

6.2 Pola kekosongan dan akar masalah

Kekosongan bersama menunjukkan asosiasi antarkolom. Pola ini dapat membantu menentukan titik investigasi, tetapi belum membuktikan bahwa input berasal dari kanal tertentu atau bahwa terjadi kegagalan pada satu sistem. Laporan ini tidak menetapkan dugaan penyebab sebagai root cause yang sudah terkonfirmasi.

6.3 Dampak terhadap metrik lanjutan

Perhitungan biaya perlu mengungkap bahwa sebagian FuelSurchargeIDR merupakan hasil imputasi. Agregasi durasi dan rating harus menyertakan jumlah pengamatan terisi agar penyebutnya jelas. Analisis keterlambatan pada Delivered perlu membandingkan durasi aktual dan rencana, bukan menghitung label No Delay.

Kolom TotalShippingCostIDR, DeliveredStatus, Route, CostPerKM, CostPerKG, dan DelayDays dibahas dalam laporan analisis sebagai transformasi notebook. Kolom-kolom tersebut tidak dibuat oleh TableCleaning.sql sehingga tidak dinyatakan sebagai keluaran cleaning SQL ini.

Sumber: TableCleaning.sql

7 BATASAN DOKUMENTASI

Penyusunan laporan hanya menggunakan tiga berkas yang dilampirkan: skrip SQL, laporan cleaning sebelumnya, dan laporan analisis. Tidak tersedia koneksi database, CSV mentah, dataset hasil cleaning, ataupun notebook untuk menjalankan ulang seluruh pemeriksaan.

Jumlah data awal, jumlah record terhapus, nilai median FuelSurchargeIDR, identitas dua order dengan biaya berbeda, dan keluaran query validasi tidak tersedia sebagai hasil eksekusi langsung. Angka 120 berasal dari komentar skrip dan laporan cleaning; angka 6.000 serta kelengkapan akhir berasal dari laporan analisis.

Penelusuran pola NULL tidak membuktikan penyebab operasional. Pemilihan biaya terendah tidak membuktikan nilai benar, dan deduplikasi berdasarkan OrderID bergantung pada asumsi granularitas. Ketepatan waktu tidak dapat disimpulkan dari status Delivered atau label No Delay saja.

Skrip tidak mendokumentasikan proses impor CSV, DDL lengkap, pemeriksaan tipe fisik, validasi semua rentang nilai, atau rekonsiliasi tanggal. Karena itu, laporan tidak menyatakan semua dimensi kualitas data telah tuntas diverifikasi.

8 KESIMPULAN

TableCleaning.sql mendokumentasikan alur pembersihan yang menghasilkan dbo.ShippingLogistics_Cleaned melalui tabel kerja. Tindakan utama mencakup imputasi alasan dan kategori, imputasi median biaya bahan bakar, pemeliharaan NULL yang bermakna, deduplikasi order, serta standardisasi nama kota.

9 LAMPIRAN TEKNIS

Salinan TableCleaning.sql berikut mempertahankan perintah dan komentar sumber. Pemenggalan baris panjang disesuaikan untuk keterbacaan. Catatan koreksi terhadap makna alias dan interpretasi komentar telah dijelaskan pada isi laporan.

```
USE LogistikDB;

SELECT TOP 10 *
FROM dbo.ShippingLogistics;

SELECT COUNT(*) AS total_OrderID_Unique
FROM dbo.ShippingLogistics;

SELECT COUNT(DISTINCT OrderID) AS total_rows
FROM dbo.ShippingLogistics;

-- duplicate table for cleaning and transformation
DROP TABLE IF EXISTS dbo.ShippingLogistics_Cleaning;

SELECT *
INTO dbo.ShippingLogistics_Cleaning
FROM dbo.ShippingLogistics;

SELECT TOP 10 *
FROM dbo.ShippingLogistics_Cleaning;

SELECT COUNT(*) AS total_rows
FROM dbo.ShippingLogistics_Cleaning;

--query to find sum of missing values in each column of the ShippingLogistics_Cleaning table
(T-SQL Query)
DECLARE @sql NVARCHAR(MAX);

SELECT @sql = STRING_AGG(
    'SELECT ''' + COLUMN_NAME + ''' AS column_name,
        COUNT(*) AS missing_count
    FROM dbo.ShippingLogistics_Cleaning
    WHERE ' + QUOTENAME(COLUMN_NAME) + ' IS NULL',
    ' UNION ALL '
)
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dbo'
    AND TABLE_NAME = 'ShippingLogistics_Cleaning';

EXEC sp_executesql @sql;

--check missing values in column DelayReason in the ShippingLogistics_Cleaning table. Why is it so
extreme?
SELECT TOP 10 DelayReason
FROM dbo.ShippingLogistics_Cleaning
GROUP BY DelayReason;

--check missing values with another column which is related to DelayReason in the
ShippingLogistics_Cleaning table
SELECT TOP 10 DeliveryStatus, DelayReason
FROM dbo.ShippingLogistics_Cleaning;

SELECT DeliveryStatus, COUNT(*) AS missing_count
FROM dbo.ShippingLogistics_Cleaning
WHERE DelayReason IS NULL
GROUP BY DeliveryStatus;
```

```

/*all missing values in DelayReason column are related to the DeliveryStatus column with status
'Delivered'. It is possible
that the missing values in DelayReason are due to the fact that the delivery was successful and
there was no delay,
hence no reason for delay was recorded. So, missing values can be handled using 'no delay'
status imputation.*/
UPDATE dbo.ShippingLogistics_Cleaning
SET DelayReason = 'No Delay'
WHERE DelayReason IS NULL AND DeliveryStatus = 'Delivered';

SELECT COUNT(*) AS delayreason_count_missing
FROM dbo.ShippingLogistics_Cleaning
WHERE DelayReason IS NULL;

--check missing values in column DeliveryDate and ActualDeliveryDays in the
ShippingLogistics_Cleaning table.
SELECT deliverydate, actualdeliverydays, deliverystatus
FROM dbo.ShippingLogistics_Cleaning
WHERE deliverydate IS NULL;

--since the item was lost, the DeliveryDate and ActualDeliveryDays fields remain NULL.
--next, we will check the missing values in the column FuelSurchargeIDR in the
ShippingLogistics_Cleaning table.
SELECT TOP 10 *
FROM dbo.ShippingLogistics_Cleaning
WHERE FuelSurchargeIDR IS NULL;

--It can be seen that, based on the top 10 data entries, FuelSurchargeIDR, VehicleType, and
PaymentMethod all have null values.
--check whether VehicleType, PaymentMethod, and FuelSurchargeIDR are NULL in the same row.
SELECT
SUM(CASE WHEN VehicleType IS NULL THEN 1 ELSE 0 END) AS VehicleType_NULL,
SUM(CASE WHEN PaymentMethod IS NULL THEN 1 ELSE 0 END) AS PaymentMethod_NULL,
SUM(CASE WHEN FuelSurchargeIDR IS NULL THEN 1 ELSE 0 END) AS FuelSurcharge_NULL,
SUM(CASE
WHEN VehicleType IS NULL
AND PaymentMethod IS NULL
AND FuelSurchargeIDR IS NULL
THEN 1 ELSE 0
END) AS Combined_NULL
FROM dbo.ShippingLogistics_Cleaning;

/*Confirmed that the missing values in VehicleType, PaymentMethod,
and FuelSurchargeIDR occur in the same 120 rows.

Handle missing values:
VehicleType and PaymentMethod are categorical variables,
so missing values are labeled as 'Unknown'.*/
UPDATE dbo.ShippingLogistics_Cleaning
SET VehicleType = 'Unknown',
PaymentMethod = 'Unknown'
WHERE VehicleType IS NULL OR PaymentMethod IS NULL;

--for FuelSurchargeIDR, we will check the missing values based on the DeliveryStatus column, which
is related to FuelSurchargeIDR.
SELECT DeliveryStatus, COUNT(*) AS missing_count
FROM dbo.ShippingLogistics_Cleaning
WHERE FuelSurchargeIDR IS NULL
GROUP BY DeliveryStatus;

--since all missing values in FuelSurchargeIDR are related to the DeliveryStatus column with

```



```

status 'Delayed', 'Delivered', and 'Returned', we can handle the missing values using median
imputation.
UPDATE dbo.ShippingLogistics_Cleaning
SET FuelSurchargeIDR = (
    SELECT DISTINCT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY FuelSurchargeIDR) OVER()
    FROM dbo.ShippingLogistics_Cleaning
    WHERE FuelSurchargeIDR IS NOT NULL
)
WHERE FuelSurchargeIDR IS NULL;

--for CustomerRating, we will stay with NULL values since it is not a mandatory field from
customer feedback and can be left as NULL.
--next, we will check the duplicate rows in the ShippingLogistics_Cleaning table.
SELECT *
FROM dbo.ShippingLogistics_Cleaning
WHERE OrderID IN (
    SELECT OrderID
    FROM dbo.ShippingLogistics_Cleaning
    GROUP BY OrderID
    HAVING COUNT(*) > 1
)
ORDER BY OrderID;

/*Duplicate OrderID analysis confirmed that most duplicate records
contain identical values across all columns and can therefore be removed.
However, two duplicate orders have significantly different ShippingCostIDR values,
indicating potential data entry errors.
For these two cases, the record with the lower ShippingCostIDR is retained
as the more reasonable value.*/
WITH DuplicateCheck AS (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY OrderID
            ORDER BY ShippingCostIDR ASC
        ) AS row_num
    FROM dbo.ShippingLogistics_Cleaning
)
DELETE FROM DuplicateCheck
WHERE row_num > 1;

SELECT OrderID, COUNT(*) AS duplicate_count
FROM dbo.ShippingLogistics_Cleaning
GROUP BY OrderID
HAVING COUNT(*) > 1;

--next, we will standardize OriginCity & DestinationCity: contains spaces and a mix of uppercase
and lowercase letters,
UPDATE dbo.ShippingLogistics_Cleaning
SET OriginCity = TRIM(UPPER(OriginCity)),
    DestinationCity = TRIM(UPPER(DestinationCity));

SELECT TOP 10 * FROM dbo.ShippingLogistics_Cleaning;

DROP TABLE IF EXISTS dbo.ShippingLogistics_Cleaned;
--create a new table with cleaned and transformed data
SELECT * INTO dbo.ShippingLogistics_Cleaned
FROM dbo.ShippingLogistics_Cleaning;

```